

How Many Bits Can an Adapter Write?

Measuring the Capacity and Memorization of Parameter-Efficient Fine-Tuning

Kaizhen Tan¹, Heqing Du², Yang Feng²

¹Carnegie Mellon University, Pittsburgh, PA, USA

²Columbia University, New York, NY, USA

Abstract

A LoRA adapter is a few megabytes that almost everyone treats as a skill rather than a record of the data behind it. We put that assumption on a scale. Extending compression-based memorization analysis to the frozen-base setting, we measure directly—in bits—how much a low-rank adapter writes into a model it never changes. The answer is both smaller than full fine-tuning and less lawful than parameter counting would predict. Adapters store a couple of bits per trainable parameter, well short of a full model’s budget; but that figure turns less on how many parameters an adapter carries than on where they sit. Move the same parameter budget from attention into the MLP and it holds nearly twice as much; strip the frozen base of its structure and the capacity all but disappears. Turned on realistic fine-tunes of Qwen2.5, the same instrument shows privacy leakage rising with the bits an adapter writes rather than the parameters it nominally has, and it draws a clean line between supervised and reinforcement learning: the secrets that supervised fine-tuning copies down verbatim, an adapter trained on verifiable rewards never records. Measuring what fine-tuning writes—rather than attacking it after the fact—turns a piece of folklore into a quantity one can design against.

Introduction

Adapter files circulate as if they were harmless diffs. LoRA adapters (Hu et al. 2022) are shared through public model hubs, exchanged as low-rank updates between federated clients, and passed between teams as task adapters—on the tacit understanding that a small file carries a skill, not the data that taught it. Nobody has checked that understanding against a measurement. The arithmetic alone should give pause: a rank-16 adapter for a 7B model holds some 4×10^7 trainable parameters, and if it stored information at the 3.6 bits per parameter that Morris et al. (2026b) found for full models, one such file could carry the equivalent of twenty megabytes of training text—a whole fine-tuning corpus of medical notes or support transcripts, passed around as a convenience.

What makes this more than a curiosity is *where* it bites. Morris et al. (2026b) showed that pretraining lives in the comfortable regime: the corpus dwarfs what the model can hold, so any single example is diluted and membership inference falls to chance. Fine-tuning inverts that picture. Adaptation sets are small and often sensitive, and they routinely hold *fewer* bits than even a modest adapter can absorb—the regime in which, by the same theory, memorization is total by

default. The one reassurance the field offers about memorization is calibrated on exactly the case that does not apply, and the only physical quantity left to bound the leak is the size of the adapter itself. Yet everything we know about that quantity comes from attacks: prior work plants secrets and tries to extract them, or runs membership inference and reports an AUC (Wang and Li 2025; Bossy et al. 2026; Malekhammadi and Farnadi 2024). An attack tells you what one adversary recovered. It does not tell you what the artifact holds.

So we weigh the artifact. Following Morris et al. (2026b), we read memorization off a compression advantage: a model that has memorized a string can describe it in fewer bits than a reference can, and the gap is the information the model absorbed. To make the reference exact we let frozen bases memorize uniformly random token sequences—strings with nothing to learn, so that every bit recovered is a bit stored, not a bit inferred. Random strings are not a use case; they are the calibration weight that lets us then trust the scale on real text.

What the scale reads back disagrees with the folklore in two directions at once. Adapters do far less than full models—a couple of bits per trainable parameter against the full-model 3.6, confirmed by fine-tuning the same base under the same budget. But the per-parameter figure is not a property of the parameters. It moves by nearly a factor of two when the same budget of weights is placed in the MLP rather than in attention, it barely notices half-precision storage, and it collapses when the frozen base has no structure to steer: the adapter that memorizes 98% of its data on a pretrained base manages 29% on a random one. What an adapter can write is set by where it writes and what it writes into, not by how many numbers it contains.

The same scale settles a second, noisier debate. A recent line argues over what reinforcement learning with verifiable rewards actually deposits in the weights—new capability, or a sharpening of what pretraining already knew (Chu et al. 2025; Morris et al. 2026a)—and argues it entirely from behavior. We bring bits instead. At matched accuracy, an RLVR adapter writes nothing our estimator can distinguish from zero, and in a direct test the private secrets that supervised fine-tuning copies down verbatim, reinforcement learning never records at all. We contribute a measurement protocol for adapter capacity, released as a harness; capacity readings

that separate written bits from parameter count, across placement, precision, and substrate; an audit tying those bits to real canary extraction and membership inference on identical checkpoints; and the first bit-level comparison of supervised against reinforcement fine-tuning. Figure 2 previews the core of it: capacity plateaus that sit well below the full-model line and move when parameter count does not.

Related Work

Measuring memorization. Early notions of memorization are operational: a string counts as memorized if a prompt elicits it (Carlini et al. 2019, 2023). Such tests cannot tell memorization from prediction, since a model may emit a string it never saw simply because the string was predictable. Morris et al. (2026b) cut the knot with compression—memorization is the bits a model saves an arithmetic coder over a reference, measured on data of known information content—and by training on uniformly random strings, where nothing is predictable, turn those saved bits into a capacity meter that reads 3.6 bits per parameter for GPT-family models. Related instruments measure or bound the same quantity through intrinsic dimension (Arnold 2025), counterfactual influence (Zhang et al. 2023), behavioral probes of fine-tuned models (Mireshghallah et al. 2022), and the memorization capacity of attention (Mahdavi, Liao, and Thrampoulidis 2024). All of it studies the full model. We carry the compression meter into the frozen-base regime, where the writable object is the adapter and the base supplies structure it never updates—and where, as it turns out, protocol choices that are harmless at full-model scale come to dominate the reading.

PEFT and privacy. A steady empirical thread reports that low-rank adaptation leaks less than full fine-tuning: under canary extraction in federated settings (Bossy et al. 2026), under extraction and membership inference in centralized ones (Wang and Li 2025), and by a differential-privacy argument that random low-rank projections sketch away gradient information (Malekmohammadi and Farnadi 2024). Attacks have since reached RLVR pipelines (Liu et al. 2025), and differentially private fine-tuning (Yu et al. 2022) remains the standard defense when leakage matters. Each of these establishes that leakage is smaller; none measures how much information the adapter actually holds, and all compare algorithms at unmatched task performance. We measure the stored information itself, on the very checkpoints we then attack.

Low-bit adaptation and what RLVR learns. Adapters have been driven from millions of parameters (Hu et al. 2022) through shared and decomposed forms (Kopiczko, Blankevoort, and Asano 2024; Bałazy et al. 2025) down to thirteen trained with GRPO (Morris et al. 2026a; Shao et al. 2024); LoRA has been shown to learn and forget less than full fine-tuning (Biderman et al. 2024) and its expressive power characterized in theory (Zeng and Lee 2024). The thirteen-parameter result carries a conjecture: supervised fine-tuning must swallow the whole token stream, while reinforcement learning’s usable signal is only the reward—a few bits per prompt—so a tiny update suffices. A parallel debate asks whether RLVR injects new ability or elicits old (Chu et al.

2025). Both are argued from behavior. We answer with the bits themselves, written into the adapter and into the policy’s likelihoods, read out at matched accuracy.

Adapters as shared artifacts. What sharpens the stakes is that adapters have become objects of exchange in their own right—published on hubs, aggregated across federated clients, and passed between teams—and every transfer moves a file whose data content no one measures. The privacy literature treats a fine-tune as a model to be attacked after training; the ecosystem treats an adapter as an asset to be shared. A capacity measured in bits speaks to both readings at once: it is what an attacker is ultimately bounded by, and what a sharer is unknowingly handing over. That is the quantity we set out to measure, and the reason a number that can be computed before training is worth more here than an attack that can only be run after it.

Measuring What an Adapter Writes

Information quantities

Let $\hat{\theta}$ be a tuned model and θ_{ref} a reference that captures what is predictable without seeing the training set. The codelength of a sequence x under a model is its negative log-likelihood in bits, $H_K(x \mid \theta) = -\log_2 p(x \mid \theta)$, which arithmetic coding realizes up to negligible overhead (Witten, Neal, and Cleary 1987). Following Morris et al. (2026b), unintended memorization is the reference codelength the tuned model undercuts,

$$\text{mem}_U(x) = H_K(x \mid \theta_{\text{ref}}) - \min(H_K(x \mid \theta_{\text{ref}}), H_K(x \mid \hat{\theta})), \quad (1)$$

summed over a dataset as $\text{mem}_U = \sum_i \text{mem}_U(x_i)$.

The idea underneath is worth stating plainly, because everything rests on it. A model that has genuinely memorized a string can predict it, and a good predictor is a good compressor—so the bits the tuned model shaves off the reference are, quite literally, the information it took in. On ordinary text that figure tangles memorization together with understanding, since a model also compresses what it correctly generalized. Random strings cut the tangle: there is nothing to understand, so every saved bit is a memorized bit, and a scale calibrated here can be trusted when we later turn it on real data.

The random-string probe makes the reference exact. Sequences are 64 uniform tokens over a 2,048-symbol vocabulary after a start token, so $H_K(x \mid \theta_{\text{ref}}) = 704$ bits by construction and nothing in x can be predicted: every bit of mem_U is a bit memorized. Fresh random sequences held out from training must then read $\text{mem}_U \approx 0$, a check that passes across our whole spine (Appendix A; appendix pointers refer to the supplement).

Two further quantities describe the artifact and the behavior. The *parameter-write* cost $\text{CL}(\Delta\theta)$ is the code-length of the trainable weights themselves—per-tensor quantized and entropy-coded, minimized over precision—and upper-bounds what shipping the adapter communicates. The *behavior-write* bits

$$W(D) = \sum_{(x, y^*) \in D} H_K(y^* \mid x, \theta_{\text{base}}) - H_K(y^* \mid x, \hat{\theta}) \quad (2)$$



Figure 1: Overview of our measurement framework. We first adapt a frozen model to uniformly random sequences, where every recovered bit must come from memorization, and then measure adapter writability, artifact codelength, and behavior-write bits before applying the same framework to privacy leakage and SFT–GRPO comparisons.

measure how far tuning moved the model’s likelihoods on canonical answers. Because W reads only the final policy, it treats a supervised and a reinforcement-learned adapter alike, though only one was trained on likelihood—the property that makes the two comparable at all. On text we report the signed excess $\Delta W = W(D_{\text{train}}) - (n_{\text{train}}/n_{\text{held}}) W(D_{\text{held}})$: the training-set bits beyond what a size-matched held-out set explains, the memorization left once ordinary learning is subtracted off.

Three readings rather than one, because they answer different questions. The artifact cost $CL(\Delta\theta)$ is what the file physically transmits—the quantity a hub or a federated aggregator would reason about when it decides what a shared checkpoint carries. The behavior bits W and their excess ΔW are what the model’s outputs reveal, and because they read a policy rather than its weights, they compare training methods that a weight-space measure could not. And mem_U , on random data, ties both to a ground truth that neither has on real text. An adapter can read high on one and low on another; the gaps between them are as much of the story as the numbers themselves.

Reading capacity cleanly

A capacity number is only meaningful once optimization, not storage, has stopped being the bottleneck—and in the adapter regime that is a delicate condition. Learning rates that are stable for full models diverge here; early-stopping rules that behave at full scale either never fire as the loss flattens or cut off exactly the over-capacity runs that define the plateau. We therefore fix a 16,000-step budget with no early stopping, sweep the learning rate and report the envelope, and place dataset sizes relative to estimated capacity so the informative

band is sampled rather than a budget-starved tail. The resulting figure is a lower bound at a declared budget: extending to 40,000 steps lifts it by 10–24% (Appendix B), so we read every value as “at least this much.” Bases are GPT-style decoders (2M and 8M non-embedding parameters) pretrained on WikiText-103 (Merity et al. 2017) then frozen; adapters are LoRA (Hu et al. 2022) and weight-tied TinyLoRA (Morris et al. 2026a). The untouched base bounds each reading from below, and full fine-tuning of the same architecture—which reaches near-total memorization under the identical budget—bounds it from above.

The Capacity of Adapters

Plateaus, far below the full-model line

Every configuration tells the same story (Figure 2): memorization climbs with the data while data is scarce, then bends onto a plateau once the dataset carries more bits than the adapter can hold. The two largest readings per configuration agree to within a few percent, and held-out sequences stay at zero throughout—so the plateau is a ceiling on storage, not an accident of the fit.

Two numbers frame the result (Table 1). Full-model training stores about 3.6 bits per parameter (Morris et al. 2026b); fine-tuning the same base under our own budget confirms it climbs past 3. Adapters, given that same budget, stop at 1.7 to 2.8. The gap is real and not an artifact of stopping early—it survives the matched budget, and the readings are stable across seeds and only creep upward with more compute, never enough to close it. An adapter, whatever else it is, is a far leakier vessel per parameter than the model it patches, but a much smaller one in absolute terms.

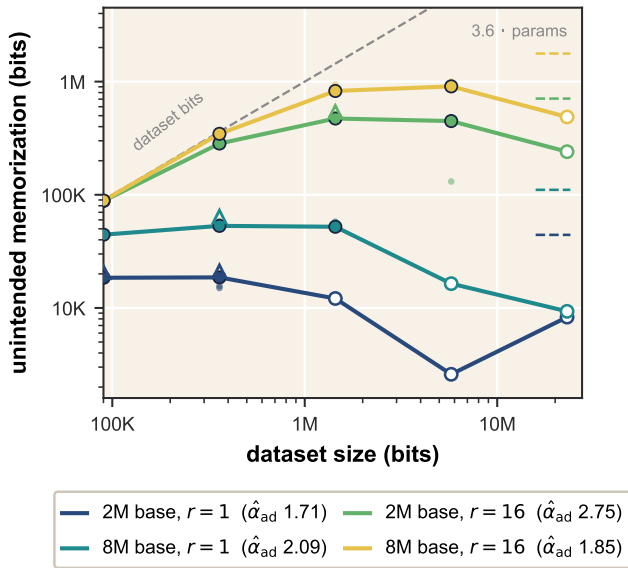


Figure 2: LoRA adapters memorize more as the dataset grows, but eventually reach a configuration-dependent plateau well below the nominal full-model capacity. The gray diagonal marks perfect memorization, the colored dashes mark 3.6 bits per full-model parameter, and hollow points indicate runs where the fixed optimization budget becomes limiting.

Past capacity the picture changes character. Push a dataset well beyond what an adapter can hold and its memorization does not simply saturate: under a fixed budget it falls, the over-capacity runs tracing a descent rather than a plateau (the hollow points of Figure 2). This is the adapter-scale shadow of the grokking transition the full-model work reports—once the data outruns the room, gradient interference under a bounded budget wins, and a model that can no longer memorize everything ends up memorizing less of it. We read this tail as an optimization regime rather than a storage reading and keep our dataset grid below it, but it marks the seam where the memorization story ends and a generalization story would begin.

It is not the parameter count

The natural objection to a capacity plateau is that it just counts parameters: more weights, more room. It does not. Fix the trainable count at about 37K and change only *where* the weights live—attention alone, all modules, or the MLP alone, with ranks chosen to keep the count equal—and per-parameter capacity runs from 1.30 to 2.43 bits, a near-doubling with parameter count held flat (Figure 3, left; three seeds, tight). The same budget of numbers stores far more when it sits in the MLP than in attention. What an adapter can hold is a property of the update’s shape, not its size.

Precision tells the same story from another angle: casting a trained adapter to bf16 costs about one percent of its memorization while halving the file, and fp16 costs essentially nothing. The bits an adapter has written are robust to how coarsely its weights are stored. Only at the extreme of weight

Configuration	Params	Peak mem _U	α_{ad}	spread
2M base, $r=1$	12,288	21.1K bits	1.71	2%
8M base, $r=1$	30,720	64.2K bits	2.09	9%
2M base, $r=16$	196,608	540K bits	2.75	3%
8M base, $r=16$	491,520	908K bits	1.85 [†]	10%
Full FT (matched)	2.2M	6.81M bits	≥ 3.07	—

Table 1: Capacity estimates for each frozen-base and LoRA-rank configuration under the same 16,000-step budget. Peak mem_U gives the largest observed memorization, $\hat{\alpha}_{\text{ad}}$ normalizes it by trainable parameter count, and spread reports variation across seeds.

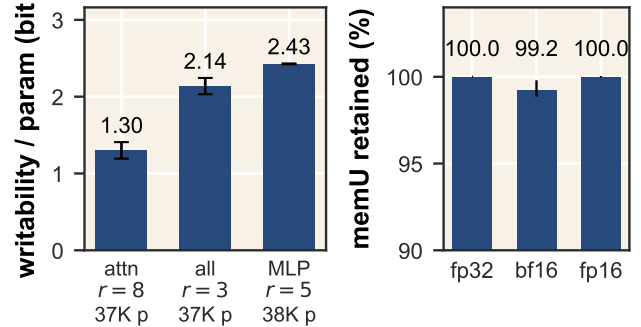


Figure 3: Adapter capacity depends more on where the trainable weights are placed than on how many there are. Left, parameter-matched MLP adapters store nearly twice as many bits per parameter as attention-only adapters; right, casting trained adapters to bf16 or fp16 preserves almost all measured memorization.

tying does the instrument hit its floor—the 4,096-parameter TinyLoRA configurations never repay the base’s own surprise at random data within the budget (a large deficit we quantify in Appendix C), so we report them as below the measurement floor rather than as zero.

And it is not the adapter alone

Hold the adapter, the optimizer, the data, and the budget fixed, and change only the frozen base underneath, and the capacity moves anyway (Figure 4). The same LoRA memorizes 29% of its data on a randomly initialized base, 79% on one pretrained on structureless synthetic text, and 98% on one pretrained on WikiText—while full fine-tuning of that random base reaches 99%, so the shortfall is specific to writing *through* frozen weights.

The two dissociations point at one mechanism. A low-rank adapter cannot move a frozen model wherever it likes; it can only push along directions the base already supports, and how much it can store is set by how rich those directions are. That is why the MLP, which carries most of a transformer’s representational width, holds more per parameter than attention, and why a base with real linguistic structure offers far more to steer than a random one. An adapter does not deposit its data into a vacuum—it re-uses what the base already knows how to represent, and its capacity is the room

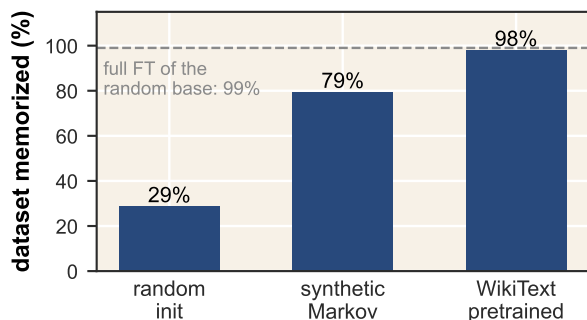


Figure 4: The same adapter writes very different amounts depending on the frozen base beneath it. Memorization rises from 29% on a random base to 98% on a WikiText-pretrained base, while full fine-tuning confirms that the random-base data itself remains memorizable.

that re-use leaves it. Capacity, then, belongs jointly to the adapter and the substrate it writes into, which is a caution as much as a finding: a bits-per-parameter number measured on one base should not be read off another.

From Bits to Leakage

A number measured on random strings earns its keep only if it predicts what a privacy audit would find on real text. So we plant secrets in a document corpus, fine-tune Qwen2.5-0.5B (Qwen Team 2024) on it, and ask whether the bits we measure line up with what an attacker can actually pull out.

The probe is the secret-sharer construction (Carlini et al. 2019): short random secrets set into a carrier corpus at controlled repetition, so that exposure—the log-rank of the true secret against ten thousand decoys—carries a known scale and a clean ceiling at rank one. Repeating a secret a handful of times, rather than once, lets us watch memorization arrive by degrees instead of as a threshold, and a held-out set matched in size keeps the behavior-bit accounting honest. It is a deliberately favorable setting for the attacker; that is the point, since we want to know whether measured capacity tracks leakage when leakage is there to be found.

The clearest read is a tale of two steps up the rank ladder (Figure 5). Going from rank 4 to rank 16 quadruples the trainable parameters but barely changes the bits the adapter writes—and exposure and membership inference barely move. Going from rank 16 to rank 64 quadruples the count again, but this time the written bits jump by half, and the secrets go with them: exposure climbs eleven bits, and the frequently repeated ones become extractable outright, ranked first among ten thousand candidates. The step that writes is the step that leaks. Parameter count, which rose the same way at both steps, does not tell the two apart.

Because rank moves bits and parameters together, we break the tie with a placement test that holds the count fixed. Three Qwen adapters of about 1.65M parameters each—rank chosen per placement to match—write different amounts by virtue of *where* they sit, and their canary exposure sorts the same way: the placement that writes the most bits leaks the most, the leanest the least, with the count held constant

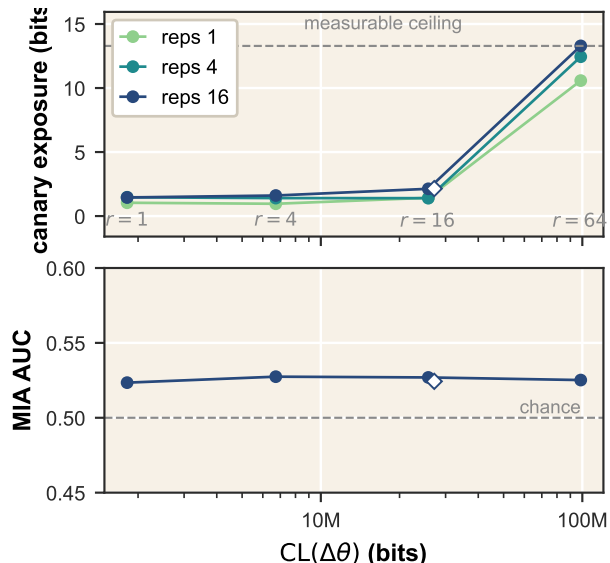


Figure 5: Canary extraction rises with the amount of information written into the adapter, while membership inference remains near chance. The top panel reports exposure for canaries repeated 1, 4, or 16 times, and the bottom panel reports loss-based membership-inference AUC for the same LoRA checkpoints.

throughout. Leakage is tracking what the adapter stored, not how large it is. (This is four planted-secret configurations plus the placement triple, not a scaling study; the full regression, with every predictor and a leave-one-out check, is in the supplement, and it neither adds nor removes the effect.)

One attack declines to cooperate, and it is worth saying why plainly. Loss-based membership inference sits at chance across the entire rank sweep, and a stronger calibrated attack (Min- $K\%$) does no better. At this corpus size, membership inference simply does not resolve the difference between these adapters—extraction does. The bits an adapter writes predict what can be pulled back out of it verbatim; whether they also predict membership at larger scale is a question our corpus is too small to answer.

For anyone who has to share the result, the reading is practical. The rank that protects privacy is not the smallest the budget allows but the smallest that still writes the data in—and the placement test adds that the same protection can come from *where* the update sits rather than how large it is. These are knobs a practitioner already turns for accuracy; the measurement gives a second reason to turn them, and a number to turn them against.

Does Reinforcement Learning Write Less?

The thirteen-parameter result of Morris et al. (2026a) rests on a claim nobody has measured: that supervised fine-tuning must absorb the whole token stream while reinforcement learning takes in only the reward, a few bits per prompt. Our behavior-write bits are defined on the final policy alone (Eq. 2), so for once the two can be weighed on the same scale.

Task	Algorithm	Steps	Acc. (%)	$W(\mathcal{D}_{tr})$ (bits)	ΔW (bits)	MIA AUC	Exp.@16 (bits)
arith	SFT	100	94.5	1,560	+106	0.501	—
arith	SFT	300	93.2 $\pm$ 0.2	1,519 $\pm$ 64	+96 $\pm$ 23	0.503 $\pm$ 0.003	—
arith	SFT	800	94.5 $\pm$ 1.0	1,644 $\pm$ 29	+271 $\pm$ 42	0.511 $\pm$ 0.001	—
arith	GRPO binary	150	85.5	1,070	-46	0.492	—
arith	GRPO binary	300	92.5 $\pm$ 0.0	1,274 $\pm$ 4	-35 $\pm$ 3	0.494 $\pm$ 0.001	—
arith	GRPO graded	300	92.0 $\pm$ 0.0	1,261 $\pm$ 8	-56 $\pm$ 9	0.490 $\pm$ 0.001	—
arith	GRPO noisy	300	86.2 $\pm$ 0.8	1,020 $\pm$ 23	-37 $\pm$ 10	0.493 $\pm$ 0.000	—
kv	SFT	300	100.0	4,879	-368	0.478	—
kv	SFT	800	100.0	4,880	-368	0.478	—
kv	GRPO binary	300	97.0	967	-269	0.454	—
kv+can	SFT	300	100.0 $\pm$ 0.0	7,640 $\pm$ 376	+2,477 $\pm$ 376	0.538 $\pm$ 0.005	12.0
kv+can	SFT	800	100.0	10,993	+5,829	0.552	13.3
kv+can	GRPO binary	300	90.8 $\pm$ 7.8	852 $\pm$ 56	-364 $\pm$ 19	0.478 $\pm$ 0.004	1.1

Table 2: Full results for SFT and GRPO on the frozen Qwen2.5-0.5B-Instruct base with rank-4 LoRA. The table reports task accuracy, behavior-write bits, excess training-set bits, membership-inference AUC, and repeated-canary exposure; values with $\pm$ summarize two seeds.

We hold the base and adapter fixed—Qwen2.5-0.5B-Instruct, LoRA rank 4—and train the two arms to the same accuracy on 3-digit arithmetic and key-value extraction, supervised on gold answers, GRPO (Shao et al. 2024) on a verifiable reward (Table 2). In the matched bin the supervised adapter carries roughly a hundred excess bits and climbs from there—by 800 steps it has written nearly three times as much, quietly compressing a training set it already answers correctly. The GRPO adapter, at the same accuracy, carries nothing: its excess is zero to within the estimator’s noise, and stays there across binary, graded, and noisy rewards alike. Supervised fine-tuning keeps writing after it has learned the task; reinforcement learning stops.

The sharpest form of the difference is a secret nobody can reach except from memory. We plant records whose answer—an eight-digit code—appears only as a supervised target, never in any prompt, so at test time it can come only from the weights. Supervised fine-tuning takes them in wholesale: repeated secrets reach rank-1 extraction among ten thousand candidates, and even a once-seen secret lands within a few bits of certainty. GRPO, trained on the identical data, leaves every one of them at chance. Excluding the planted records from the accounting drops the supervised adapter’s excess from roughly five thousand bits to essentially nothing—its entire measured surplus *is* the secrets, not a general habit of memorizing the task. The mechanism is plain: a verifiable reward pays out only when the policy happens to produce the exact secret, which the base never will, so no gradient ever points toward it, while supervision writes it down by construction. And the asymmetry is not merely the impossibility of that gradient—made the secret learnable, a two-digit code a handful of samples can stumble onto, GRPO learns the task to 90% accuracy and *still* declines to store the secret that supervision copies down.

This bears directly on the elicitation-versus-injection debate. If reinforcement learning were quietly injecting new content into the weights, a secret it was trained to reproduce is exactly the content that would show up—and it does not.

What the reward can do is raise or lower the probability of behavior the base can already produce; what it cannot do is deposit a fact the base would never sample on its own. Supervised fine-tuning has no such limit, and writes the fact down the first time it sees it. The bit-level picture is thus consistent with the elicitation reading of RLVR and hard to reconcile with strong injection, at least for the kind of content memorization would carry.

Two honest edges. The secret test is a stress test, not an accuracy-matched comparison—supervised fine-tuning reaches perfect accuracy where GRPO spans the high nineties—so it shows what each method *does* with a memorable target, not a controlled bit rate. And the reward-entropy dial the conjecture predicts would move things, does not: richer rewards do not measurably raise what GRPO writes, because on these tasks it operates so far below any entropy bound that the bound is slack. What the measurements do settle is the direction the debate has been arguing from behavior alone—reinforcement learning, at least here, deposits into the weights almost nothing of what it is shown.

Discussion

An adapter’s capacity is a number you can put on it. Weighed in bits, a LoRA file holds a couple of bits per trainable parameter—less than a full model, but far from the nothing its reputation as a harmless diff suggests—and how much it holds is decided as much by where the update sits and what base it writes through as by how large it is. That reframes the privacy question a shared adapter raises. Instead of training first and attacking afterward to find out what leaked, one can weigh an adaptation set’s information content against the adapter’s measured capacity beforehand, and know which regime one is in.

The check is concrete. Take the rank-16 adapter for a 7B model from our opening: at the two to three bits per parameter we measure, it holds on the order of ten megabytes of text—several thousand pages, more than most fine-tuning corpora contain. Set that capacity against the corpus. A corpus that

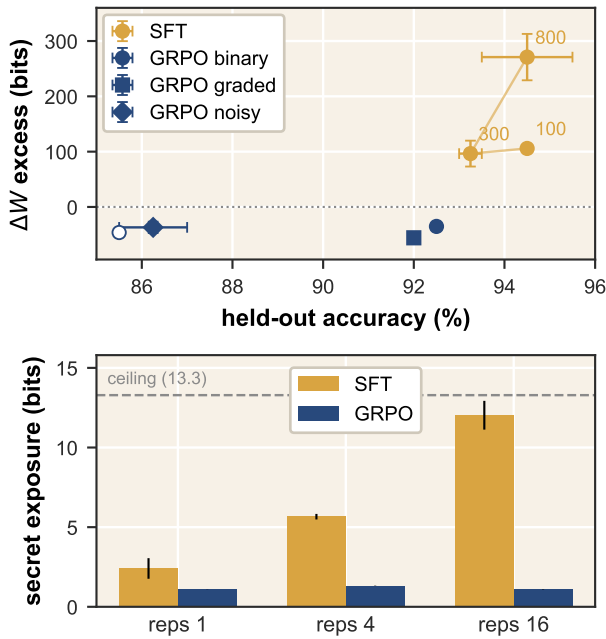


Figure 6: SFT continues to write training-specific information after reaching high task accuracy, whereas GRPO remains near zero on the same measure. The bottom panel sharpens this contrast: SFT memorizes planted secrets, while GRPO leaves their exposure close to chance.

carries fewer bits than the adapter can hold sits squarely in the regime where, by the theory we build on, memorization is total, and the resulting file should be treated as carrying its training data until an audit shows otherwise; a corpus that carries far more dilutes any single example and pulls the risk back down. The ratio of dataset bits to adapter capacity—both computable before a single gradient step—places a fine-tune on that spectrum in advance. And the levers that move it are exactly the ones no one currently adjusts for privacy: not the rank alone, but where the adapter is placed and how much structure the frozen base already carries.

That last point changes what it means to share an adapter at all. A hub hosting a LoRA, a federated client uploading an update, a marketplace trading a task adapter—each is, on this accounting, distributing a lossy but often complete copy of whatever data the adapter was small enough to memorize and large enough to hold. Treating adapters as data artifacts rather than skill patches, and publishing a measured capacity alongside a checkpoint the way a file lists its size, would let whoever downloads it reason about exposure before they ever run an attack. It would also give the party doing the fine-tuning something to act on: a target capacity to design for, met by choosing a rank, a placement, and a base whose measured room sits below the information the data must not reveal, rather than a hope that a small file will keep a secret it was never too small to hold.

The same instrument speaks to a live argument it was not built for. Whether reinforcement learning with verifiable rewards teaches a model anything new, or merely sharpens

what pretraining left, has been fought almost entirely with behavioral evidence. Reading the bits directly gives a different kind of answer: at matched accuracy the reinforcement-learned adapter writes essentially nothing, and the private facts supervised fine-tuning memorizes verbatim, it never records. Taken with the direction of the thirteen-parameter result, this is a reason to reach for verifiable-reward training precisely where an adapter will be shared and its training data must not travel with it—and a caution that the same is not true of supervised fine-tuning, which will carry the data along by construction.

The accounting is conservative by construction. Every capacity we report is a lower bound at a declared optimization budget, so a measurement can understate what an adapter holds but never inflate it—the safe direction for a privacy claim—and the protocol runs unchanged from a few-million-parameter base up to 0.5B, where it returns the same picture. The random-string probe, in turn, is a calibration instrument rather than a threat model: it isolates storage by stripping away everything a model could otherwise learn, which is precisely why the text audits of Sections and carry the leakage argument on real data.

Conclusion

Weighed in bits, a parameter-efficient adapter is not the harmless diff its reputation suggests: it holds a couple of bits per trainable parameter, and its capacity is a property of the update’s shape and the substrate it writes through, not of parameter count alone. That turns the privacy of a shared adapter into something one can measure in advance rather than discover only under attack—set the information in an adaptation set against the adapter’s capacity, and the regime is fixed before the first gradient step. Read at the same bit level, reinforcement learning with verifiable rewards deposits almost nothing into the weights where supervised fine-tuning writes its data down verbatim, settling with direct evidence a question argued until now from behavior alone.

What we mean to leave behind is the method. The harness we release measures, in bits and on any frozen base, what an adapter has written, and the three readings it returns are cheap enough to run before a checkpoint is shared, not only after it is attacked. What transfers, at any scale, is that move: a scale you can hang on an adapter, and the habit of asking not what an attacker can recover from a fine-tune, but what the fine-tune was ever able to write.

References

- Arnold, S. 2025. Memorization in Language Models through the Lens of Intrinsic Dimension. In *Proceedings of the First Workshop on Large Language Model Memorization (L2M2)*, 23–28. Association for Computational Linguistics.
- Bařazy, K.; Banaei, M.; Aberer, K.; and Tabor, J. 2025. LoRA-XS: Low-Rank Adaptation with Extremely Small Number of Parameters. In *ECAI 2025: 28th European Conference on Artificial Intelligence*, 3194–3201. IOS Press.
- Biderman, D.; Portes, J.; Ortiz, J. J. G.; Paul, M.; Greengard, P.; Jennings, C.; King, D.; Havens, S.; Chiley, V.; Frankle, J.; Blakeney, C.; and Cunningham, J. P. 2024. LoRA Learns

- Less and Forgets Less. *Transactions on Machine Learning Research*.
- Bossy, T.; Vignoud, J.; Rabbani, T.; Troncoso Pastoriza, J. R.; and Jaggi, M. 2026. Mitigating Unintended Memorization with LoRA in Federated Learning for LLMs. *Transactions on Machine Learning Research*.
- Carlini, N.; Ippolito, D.; Jagielski, M.; Lee, K.; Tramèr, F.; and Zhang, C. 2023. Quantifying Memorization Across Neural Language Models. In *International Conference on Learning Representations (ICLR)*.
- Carlini, N.; Liu, C.; Erlingsson, Ú.; Kos, J.; and Song, D. 2019. The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks. In *28th USENIX Security Symposium*.
- Chu, T.; Zhai, Y.; Yang, J.; Tong, S.; Xie, S.; Schuurmans, D.; Le, Q. V.; Levine, S.; and Ma, Y. 2025. SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training. In *Proceedings of the 42nd International Conference on Machine Learning*, 10818–10838. PMLR.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- Kopiczko, D. J.; Blankevoort, T.; and Asano, Y. M. 2024. VeRA: Vector-based Random Matrix Adaptation. In *International Conference on Learning Representations (ICLR)*.
- Liu, Y.; Zhang, H.; Zheng, J.; Sun, Z.; Peng, Z.; Wei, J.; Cong, T.; Yang, Y.; and He, X. 2025. Auditing Data Membership in Reinforcement Learning With Verifiable Rewards. arXiv preprint arXiv:2511.14045.
- Mahdavi, S.; Liao, R.; and Thrampoulidis, C. 2024. Memorization Capacity of Multi-Head Attention in Transformers. In *International Conference on Learning Representations (ICLR)*.
- Malekmohammadi, S.; and Farnadi, G. 2024. LoRA Provides Differential Privacy by Design via Random Sketching. arXiv preprint arXiv:2409.17538.
- Merity, S.; Xiong, C.; Bradbury, J.; and Socher, R. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations (ICLR)*.
- Mireshghallah, F.; Uniyal, A.; Wang, T.; Evans, D.; and Berg-Kirkpatrick, T. 2022. An Empirical Analysis of Memorization in Fine-tuned Autoregressive Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1816–1826. Association for Computational Linguistics.
- Morris, J. X.; Mireshghallah, N.; Ibrahim, M.; and Mahlouljifar, S. 2026a. Learning to Reason in 13 Parameters. arXiv preprint arXiv:2602.04118.
- Morris, J. X.; Sitawarin, C.; Guo, C.; Kokhlikyan, N.; Suh, G. E.; Rush, A. M.; Chaudhuri, K.; and Mahlouljifar, S. 2026b. How Much Do Language Models Memorize? In *Proceedings of the 43rd International Conference on Machine Learning*. PMLR.
- Qwen Team. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300.
- Wang, F.; and Li, B. 2025. Leaner Training, Lower Leakage: Revisiting Memorization in LLM Fine-Tuning with LoRA. arXiv preprint arXiv:2506.20856.
- Witten, I. H.; Neal, R. M.; and Cleary, J. G. 1987. Arithmetic Coding for Data Compression. *Communications of the ACM*, 30(6): 520–540.
- Yu, D.; Naik, S.; Backurs, A.; Gopi, S.; Inan, H. A.; Kamath, G.; Kulkarni, J.; Lee, Y. T.; Manoel, A.; Wutschitz, L.; Yekhanin, S.; and Zhang, H. 2022. Differentially Private Fine-tuning of Language Models. In *International Conference on Learning Representations (ICLR)*.
- Zeng, Y.; and Lee, K. 2024. The Expressive Power of Low-Rank Adaptation. In *International Conference on Learning Representations (ICLR)*.
- Zhang, C.; Ippolito, D.; Lee, K.; Jagielski, M.; Tramèr, F.; and Carlini, N. 2023. Counterfactual Memorization in Neural Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.